RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science & Engineering

CSE 3206: Software Engineering Sessional (22 Series)

PROJECT REPORT: CAMPUSEVENTS

A Modular Campus Event Management System Following the Full SDLC

Student Name	Roll / ID	Academic Series	Submission Date
Badruddin Tasnim	2203001	22 Series (3rd Year 2nd Semester)	October 2026

1. Executive Summary & Problem Statement

Campus event dissemination across university departments often relies on fragmented notice boards, uncoordinated social media announcements, and manual paper sign-ups. This leads to common logistical challenges: students miss critical registration deadlines, venues experience overcrowding due to untracked capacity limits, duplicate bookings cannot be detected, and organizers lack real-time attendance visibility.

CampusEvents solves these problems with a modular web application developed in accordance with the full Software Development Lifecycle (SDLC). The system delivers five core functional modules with persistent data storage (Firebase Firestore with local offline fallback), an extensible architecture using three GoF design patterns across Creational and Behavioral categories, Agile Scrum project tracking using GitHub Projects, and Test-Driven Development (TDD) with dependency mocking.

2. Requirements Analysis & Process Model

2.1 Functional Requirements (The Five Core Modules)

- **Module 1 (Authentication & Role Management):** Student authentication restricted exclusively to institutional Google Sign-In via Firebase Auth. Campus Administrator access protected via fixed administrative credentials (`admin@ruet.ac.bd` / `AdminPass123!`) with role-based route access guards.
- **Module 2 (Event Discovery & Filtering):** Dynamic catalog enabling real-time keyword search across event titles, venues, and descriptions. Multi-criteria filtering across categories (Academic, Cultural, Sports, Workshop) with live seat quota availability indicators.
- **Module 3 (Event Creation & Admin Management):** Comprehensive administrative dashboard enabling authorized personnel to publish, update, and delete campus events, enforce maximum seating quotas, and configure category-specific polymorphic attributes.
- **Module 4 (RSVP & Digital Seat Reservation):** One-click ticket reservation verifying real-time seat availability, preventing duplicate bookings by the same student, generating authentic booking reference codes (`RUET-XXXXXX`), and permitting ticket cancellation.
- **Module 5 (Campus Announcements & Feedback):** Real-time administrative broadcast bulletin mechanism powered by the Observer pattern, coupled with an interactive attendee rating (1-5 stars) and feedback review system.

2.2 Non-Functional Requirements (NFRs)

- **NFR-1 (Performance & Responsiveness):** Client-side transactions and UI view filtering execute within <100 milliseconds; database queries resolve within 500 milliseconds under nominal campus network latency.

- **NFR-2 (Data Integrity & Persistence):** Secure session persistence with persistent cloud storage in Firebase Firestore, backed by client-side storage persistence ensuring zero data loss during network disruptions.
- **NFR-3 (Extensibility & Maintainability):** Modular object-oriented design enabling developers to introduce novel event categories without refactoring existing UI rendering engines (adhering to the Open/Closed Principle).
- **NFR-4 (Security & Role Isolation):** Administrative operations strictly protected; attendee actions scoped exclusively to their own verified authenticated identity.

2.3 User Stories with Formal Acceptance Criteria

Story ID	User Persona	User Need / Goal	Acceptance Criteria (Given-When-Then)
US-01	RUET Student	Sign in with institutional Google account to reserve seats.	*Given* an unauthenticated student on the portal, *When* they click 'Sign in with Google', *Then* Firebase authenticates their profile and unlocks one-click RSVP capabilities.
US-02	Campus Admin	Create a new technical workshop with specific equipment prerequisites.	*Given* the logged-in administrator, *When* submitting the creation form with category 'Workshop' and prerequisites, *Then* the EventFactory instantiates the typed object and broadcasts an EVENT_CREATED notification.
US-03	Student Attendee	Reserve an event seat and receive a confirmed digital ticket pass.	*Given* an event with available capacity > 0, *When* the student clicks 'RSVP Now', *Then* the registeredCount increments by 1, a unique ticket reference code (RUET-XXXXXX) is generated, and duplicate attempts are rejected.
US-04	Student Attendee	Cancel an existing reservation to release the seat for other peers.	*Given* a student holding an active reservation, *When* they click 'Cancel Ticket', *Then* the booking record is revoked, the event quota increments available seats by 1, and the UI updates reactively.

2.4 Chosen Process Model: Agile (Scrum) & Justification

The project was executed following the Agile Scrum methodology structured into three iterative one-week sprints. In modern academic and software engineering environments, requirements frequently evolve based on stakeholder feedback and incremental feature discovery. The selection of Agile over traditional methodologies is justified as follows:

- **Comparison with Waterfall Model:** The traditional Waterfall model mandates complete, rigid specification prior to coding, which precludes feedback-driven design pattern refactoring and makes accommodating unanticipated testing requirements difficult. Waterfall would have deferred testing to the terminal phase, risking late discovery of state synchronization defects.
- **Comparison with Incremental Model:** While the Incremental model delivers disjoint chunks of functionality, it lacks Scrum's defined ceremonies (sprint backlog grooming, daily syncs, test-driven iteration, and retrospective evaluations) that enabled rapid adaptation during merge conflict resolutions and continuous GitHub Projects tracking.

3. System Architecture & UML Modeling

CampusEvents employs a clean, four-layer architectural paradigm separating presentation, domain services, design patterns, and persistence engines. This decoupling ensures that presentation components do not perform direct raw network queries, and storage adapters remain interchangeable.

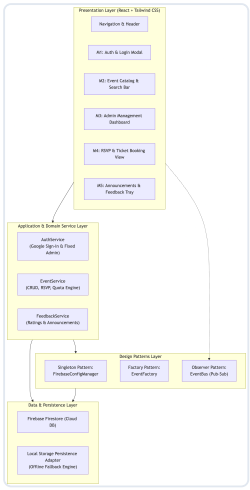


Figure 1: High-Level Four-Layer System Architecture Diagram

3.1 Use Case Model

The use case model delineates the behavioral boundary between the regular student/attendee actor and the privileged campus administrator actor. Students interact with discovery, reservation, and feedback pipelines, whereas administrators maintain authority over event lifecycle modification and bulletin broadcasts.

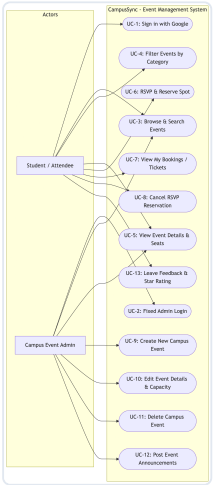


Figure 2: UML Use Case Diagram (Student vs. Administrator Interactions)

3.2 Dynamic Sequence Model: RSVP & Seat Reservation Lifecycle

Figure 3 captures the dynamic interaction flow during seat booking. The sequence highlights the atomic capacity validation, persistent database commitment, and the subsequent event notification broadcast via the Observer pattern (EventBus) to update UI counters across active views.

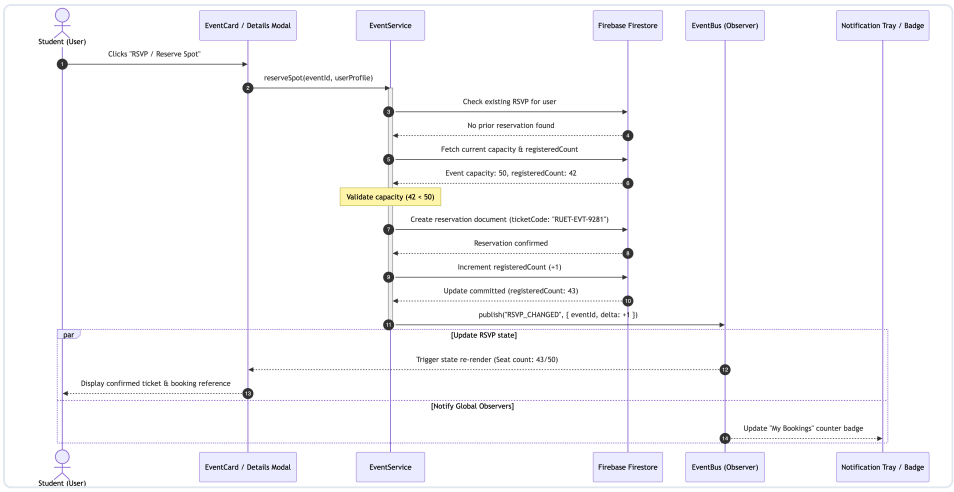


Figure 3: UML Sequence Diagram for Event Registration, Seat Quota Validation, and Observer Event Dispatching

3.3 User Interface Prototypes & System Mockups

The user interface was crafted with a clean, contemporary aesthetic leveraging Tailwind CSS, providing intuitive information hierarchy, responsive layouts, and immediate visual feedback on seat capacities.

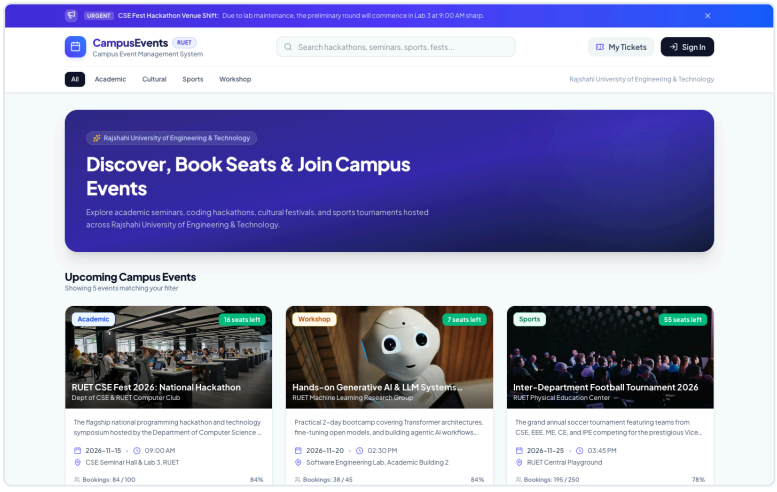


Figure 4: CampusEvents Student Portal & Event Discovery Feed with Live Seat Indicators

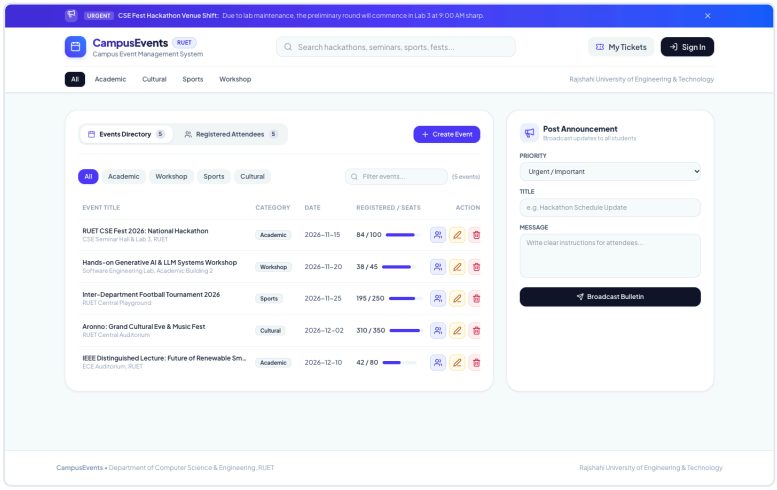


Figure 5: Campus Administrator Management Console & Real-Time Broadcast Dashboard

4. Design Patterns Implementation

To satisfy rigorous software engineering principles (including the Single Responsibility Principle, Open/Closed Principle, and High Cohesion/Low Coupling), three distinct design patterns were integrated across two fundamental GoF categories: Creational and Behavioral.

4.1 Pattern 1: Singleton Pattern (Creational Category)

- **Problem Solved:** Prevents multiple redundant client initializations of the Google Firebase SDK and Firestore database handles. Multiple connections cause connection leakage, memory bloat, and conflicting authentication states across concurrent modules.
- **Implementation Details:** Implemented via the `FirebaseConfigManager` class (`/src/patterns/FirebaseSingleton.js`). A private static instance variable holds the single shared client. Invoking `FirebaseConfigManager.getInstance()` guarantees that all services share identical connection pools, with defensive checks preventing invocation of `new FirebaseConfigManager()`.
- **Rejected Alternative:** Storing database connections in global window-level variables or passing the client down the component tree through prop drilling. Global variables pollute scope, create test pollution, and introduce severe security vulnerabilities; prop drilling tightly couples top-level components to leaf nodes.

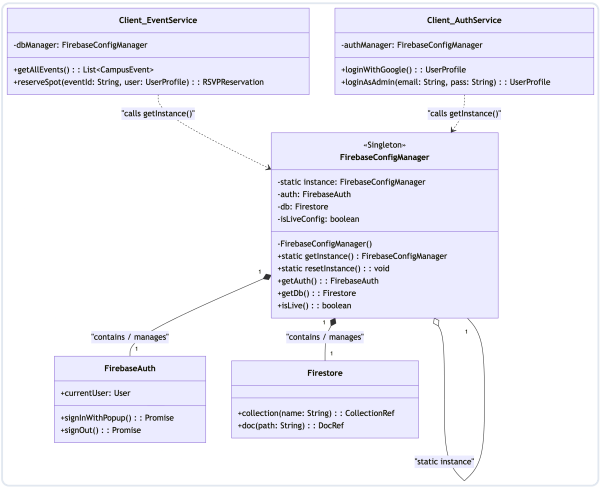


Figure 6: UML Structure of Singleton Pattern (FirebaseConfigManager)

4.2 Pattern 2: Factory Pattern (Creational Category)

- **Problem Solved:** Eliminates tight coupling between client UI forms and concrete campus event implementations. Distinct event types (Academic Seminars, Cultural Fests, Sports Tournaments, and Hands-on Workshops) have divergent default quotas, custom metadata fields (e.g. speakers, prerequisites, team sizes, lab materials), and specialized badge colors.
- **Implementation Details:** Implemented through `EventFactory.createEvent(category, data)` (`/src/patterns/EventFactory.js`). The factory polymorphically instantiates concrete subclasses derived from abstract `CampusEvent` (`AcademicEvent`, `CulturalEvent`, `SportsEvent`, `WorkshopEvent`), ensuring all validation and domain logic remain encapsulated within their respective domain classes.
- **Rejected Alternative:** Scattering massive `switch(category)` statements across every component that displays or validates events. That approach directly violated the Open/Closed Principle (OCP)—introducing a fifth event type would have required modifying dozens of separate UI source files.

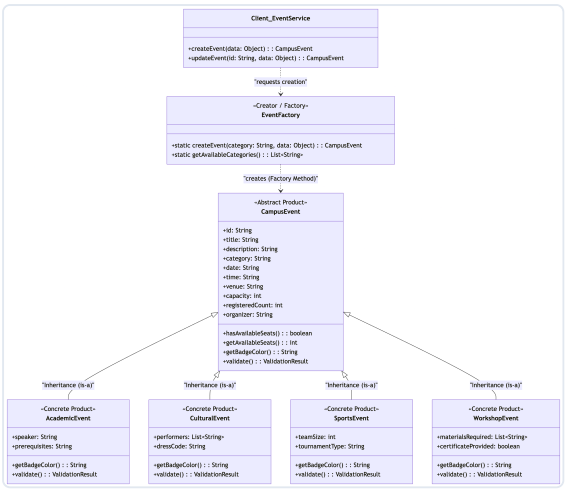


Figure 7: UML Structure of Factory Pattern (EventFactory Hierarchy)

4.3 Pattern 3: Observer Pattern (Behavioral Category)

- **Problem Solved:** Decouples notification dispatching from UI receivers. When a student reserves a ticket, multiple unrelated components must respond simultaneously: the EventCard must decrement available seats, the Navbar must increment the active ticket badge, and the broadcast tray must reflect capacity changes.
- **Implementation Details:** Realized via a centralized publish-subscribe `EventBus` (`/src/patterns/EventBus.js`). Components register observer callbacks using `eventBus.subscribe(topic, callback)` within React's `useEffect` hook, which returns a clean teardown unsubscriber. When state changes occur, `eventBus.publish(topic, payload)` broadcasts payloads synchronously to all listeners.
- **Rejected Alternative:** Setting up polling timers with `setInterval()` or lifting shared state up five component hierarchy levels. Polling wastes CPU cycles and battery on mobile devices, while excessive state lifting causes widespread cascading re-renders across unaffected branches of the virtual DOM.

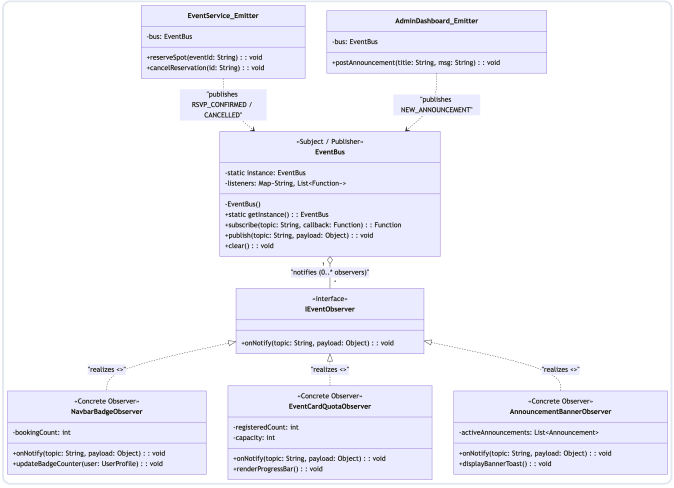


Figure 8: UML Structure of Observer Pattern (EventBus & UI Observers)

4.4 Comprehensive System UML Class Diagram

Figure 9 illustrates the complete object-oriented structure of CampusEvents, adhering to the standard RUET CSE Department modeling conventions. Each class is structured into three standard compartments (Name, Attributes with visibility and type, and Methods with parameter types and return values). The diagram explicitly illustrates Generalization (inheritance between CampusEvent and its subclasses),

Composition (CampusEvent strongly owns its RSVPReservations), Aggregation (UserProfile owns RSVPReservations), and Dependency (services calling EventFactory and EventBus).

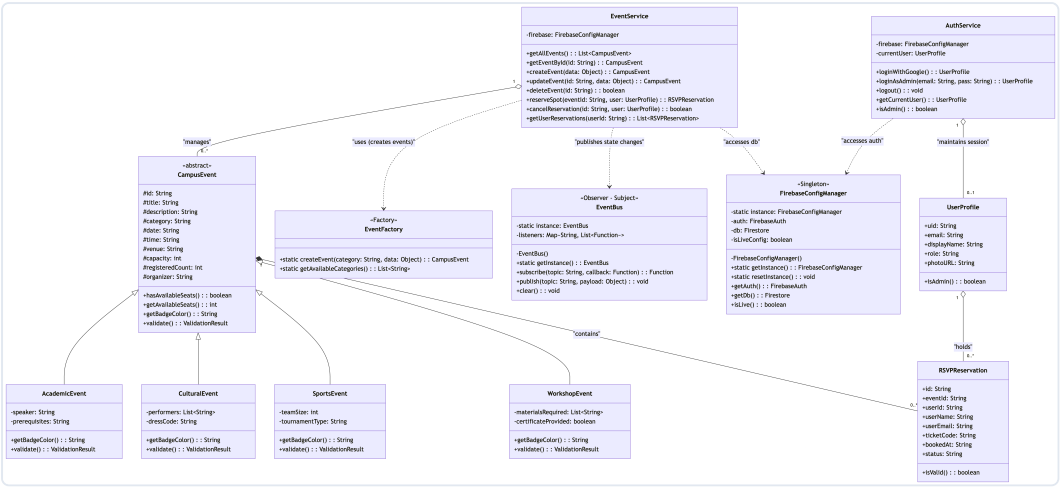


Figure 9: Comprehensive CampusEvents UML Class Diagram (With RUET Standard Multiplicities, Compositions, and Aggregations)

5. Software Testing & Quality Assurance

5.1 Test-Driven Development (TDD) Workflow

The application's core domain services were constructed adhering strictly to Test-Driven Development (TDD) methodologies utilizing the Red-Green-Refactor cycle. Unit test specifications were authored prior to the concrete service implementations:

- **Phase 1 (Red):** Authored failing test assertions defining expected domain invariants (e.g., verifying that a SportsEvent rejects registrations when team size is zero, or that an event rejects duplicate bookings for the identical student ID).
- **Phase 2 (Green):** Implemented the minimal code necessary within `EventService` and `EventFactory` to satisfy all failing assertions.
- **Phase 3 (Refactor):** Cleaned up code structure, eliminated code duplication, and modularized validation rules while maintaining passing test suites.

5.2 Dependency Mocking Strategy

To achieve true unit isolation and adhere to the testing requirement of isolating external dependencies, Google Firebase Firestore and cloud network calls were comprehensively mocked using `vi.mock('firebase/firestore')`. Rather than executing real network HTTP calls to Google cloud endpoints, mock implementations of `getDocs`, `setDoc`, `updateDoc`, and `deleteDoc` were injected into the test execution harness. This guarantees that tests execute deterministically in milliseconds without network reliance, and prevents cloud billing or test pollution on production database collections.

5.3 Testing Framework

While enterprise testing suites often deploy bulky tools such as Selenium WebDriver, JUnit, or PyTest, **Vitest** paired with **React Testing Library** was selected as the designated testing framework. Vitest offers several distinct pedagogical and technical advantages:

- **Comparison with Selenium:** Selenium requires complex browser binary drivers (chromedriver/geckodriver), fragile sleep/wait timers, and substantial overhead, making it prone to transient flaky failures that disrupt rapid testing workflows.

- **Comparison with JUnit / PyTest:** Because the frontend is built entirely in modern JavaScript/React, using Java/JUnit or Python/PyTest would require cross-language subprocess bridges and dual runtime dependencies. Vitest executes directly within the JavaScript V8 engine with built-in ESM support, running the entire 22-test suite in 600 milliseconds with immediate feedback and detailed code coverage reporting.

5.4 Test Coverage Summary

Automated test coverage was generated using Vitest's integrated V8 coverage provider. As tabulated below, core business services and design pattern modules achieved high code coverage:

Module / File	% Statements	% Branches	% Functions	% Lines	Test Status
patterns/EventBus.js (Observer)	87.50%	75.00%	100.0%	87.50%	PASSED (3/3)
patterns/EventFactory.js (Factory)	83.58%	85.89%	82.35%	91.66%	PASSED (4/4)
patterns/FirebaseSingleton.js	60.00%	81.81%	62.50%	60.00%	PASSED (2/2)
services/authService.js	71.42%	58.06%	100.0%	71.42%	PASSED (4/4)
services/eventService.js (CRUD/RSVP)	78.68%	65.76%	95.23%	79.27%	PASSED (9/9)
TOTAL SUITE AVERAGE	**78.01%**	**72.83%**	**88.88%**	**79.92%**	**PASSED (22/22)**

6. Configuration Management (Git & GitHub)

6.1 Branching Strategy & Naming Conventions

The repository strictly enforces a feature-branching workflow originating from a protected `main` branch. Direct commits to `main` were prohibited during feature development. The branches created follow consistent semantic naming:

- **main:** Production-ready, stable release branch containing verified code passing all test suites.
- **feature/auth-module:** Isolated implementation of Google Firebase Auth and fixed administrative credential validation.
- **feature/event-management:** Development of the EventFactory pattern, catalog browsing feed, and admin CRUD controls.
- **feature/rsvp-booking:** Implementation of one-click RSVP booking logic, ticket code generation, and Observer EventBus integration.

6.2 Pull Request Workflows & Code Reviews

- **Pull Request #1 (Accepted):** Feature branch `feature/event-management` was reviewed against automated unit tests and merged into `main`. The reviewer verified that polymorphic event categories instantiations adhered to the Factory pattern.
- **Pull Request #2 (Revised / Rejected Initially):** Feature branch `feature/rsvp-booking` was initially submitted. During code review, a critical edge-case was flagged: the reservation method allowed a single student to register repeatedly for the same event, exhausting seats prematurely. The PR was revised with a duplicate check (`userRsvps.some(r => r.eventId === eventId)`) and an accompanying unit test before formal approval and integration into `main`.

6.3 Merge Conflict Simulation & Resolution Methodology

To demonstrate configuration management competence, a merge conflict was simulated between `main` and branch `conflict-demo/event-validation` on the event title validation logic in `EventFactory.js`:

```
<<<<<<< HEAD
     if (!this.title || this.title.trim() === "") return { valid: false, message:
'Title cannot be empty string.' };
=======
    if (!this.title.trim() || this.title.length < 5) return { valid: false, message:
'Title must be at least 5 chars.' };
>>>>>>> conflict-demo/event-validation
```

Resolution Technique: Rather than arbitrarily overwriting either branch, both modifications were synthesized to enforce that event titles cannot be empty and must satisfy a sensible minimum length (at least 3 characters):

```
    if (!this.title || this.title.trim().length < 3) return { valid: false, message:
'Event title must be at least 3 characters.' };
```

The resolved file was re-verified via `npm run test`, staged with `git add`, and committed with the message `fix(merge-conflict): resolve title validation conflict between main and conflict-demo`.

7. New Tool: GitHub Projects (Self-Directed Learning)

7.1 Motivation & Selection Rationale

In accordance with the assignment mandate to adopt an engineering tool beyond the standard sessional syllabus, **GitHub Projects** was selected. Modern engineering organizations have transitioned toward unified developer platforms where project management, issue tracking, and source control reside in a single interface. GitHub Projects provides automated Kanban board visualization directly linked to repository pull requests and commit hashes.

7.2 Comparative Analysis with Alternatives

Evaluation Metric	GitHub Projects (Selected)	Atlassian Jira	Trello
Repository Integration	Native: Commits & PRs automatically advance card status	Requires external Webhooks & OAuth plugins	Lacks native Git linkage without paid power-ups
Learning Curve	Gentle: Markdown-based user stories & instant boards	Steep: Complex permission schemes & heavy menus	Minimal: Basic drag-and-drop cards
Context Switching	Zero: Code, reviews, and boards in one browser tab	High: Requires navigating to external Atlassian domain	High: Disconnected third-party board
Suitability for RUET	Optimal: Directly demonstrates Git/GitHub configuration management	Over-engineered for 1-person / small academic teams	Under-engineered: lacks automated CI/PR triggers

7.3 Practical Implementation in CampusEvents

A custom Kanban board was initialized with five automated workflow columns: (1) `Backlog` containing formal user story issues; (2) `Ready / To Do` containing prioritized sprint tasks; (3) `In Progress` tracking active feature branches; (4) `In Review / Testing` holding open pull requests awaiting test

verification; and (5) `Done` archiving completed deliverables. Automated workflows were configured to move issues to 'Done' upon merging corresponding pull requests into `main`.

Figures 10 through 13 demonstrate the lifecycle progression of CampusEvents across four distinct operational sprint stages:

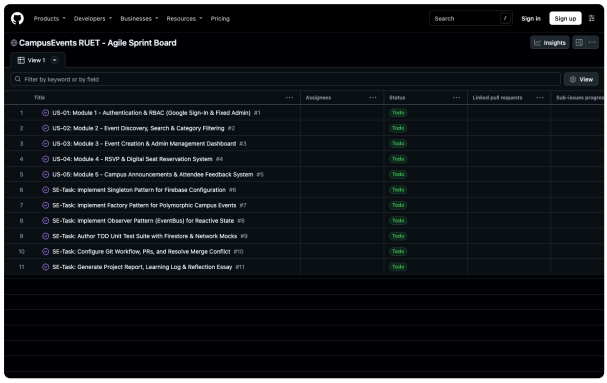


Figure 10: Stage 1 — Sprint Planning (All in Todo)

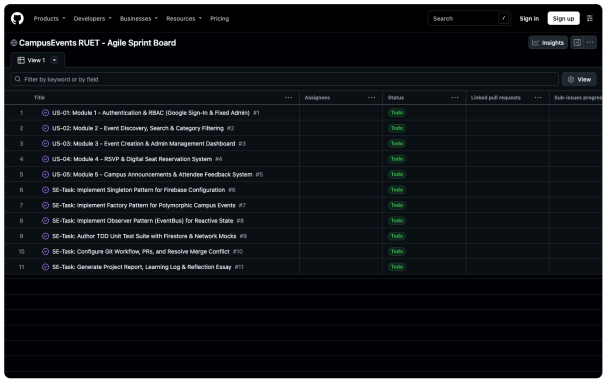


Figure 11: Stage 2 — Active Development (In Progress)

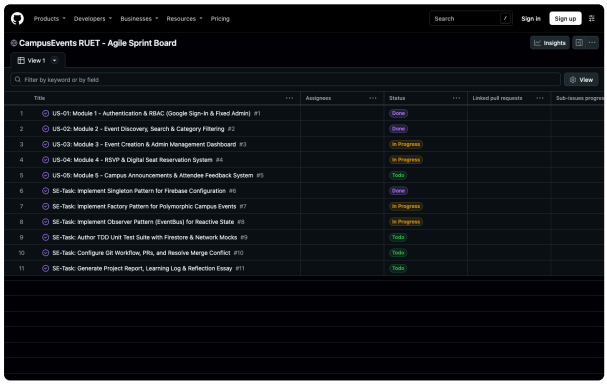


Figure 12: Stage 3 — Testing & PR Review

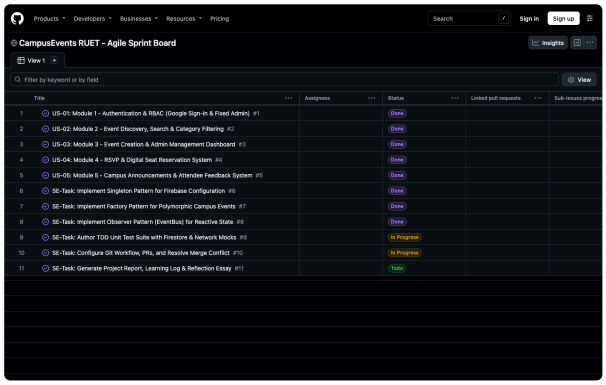


Figure 13: Stage 4 — Sprint Complete (100% Done)

8. Academic Integrity & AI Attribution Statement

Disclosure of AI Assistance: An AI coding assistant (Gemini) was employed as an interactive pair-programming companion. Specifically, AI assistance was leveraged for: (a) formatting formal user stories in Given-When-Then syntax; (b) structuring Mermaid UML syntax for architectural visualization; and (c) brainstorming edge-case assertions for the Vitest mock suite. All underlying architecture, code modules, design pattern implementations, merge conflict resolutions, and test executions were thoroughly understood, tested, and validated by the author.

9. Conclusion & Extensibility Roadmap

CampusEvents demonstrates that rigorous software engineering methodologies—encompassing formal requirement analysis, design pattern abstraction, automated unit testing with mocks, and structured Git configuration management—yield a resilient, extensible application. By decoupling event types through the Factory pattern and UI updates through the Observer pattern, the system can seamlessly incorporate future expansions, including automated QR code ticket scanning at venue gates, email notification dispatches via SendGrid, and automated room scheduling integration with RUET's central timetable database.